1 Write a Python program to create a class Car and demonstrate

encapsulation.

Aim

To write a Python program that defines a Car class and demonstrates encapsulation by using private attributes and public methods to access or modify them safely.

⚙ Algorithm

1. Define a class .
2. Use private attributes (e.g., , ,) to hide data.
3. Provide getter methods to access private attributes.
4. Provide setter methods to update private attributes with validation.
5. Create an object of and demonstrate encapsulation by accessing/modifying data only through methods.

CODE

```
# Program to demonstrate encapsulation in Python


class Car:
    def __init__(self, brand, model, price):
        # Private attributes
        self.__brand = brand
        self.__model = model
        self.__price = price


    # Getter methods
    def get_brand(self):
        return self.__brand


    def get_model(self):
        return self.__model
```

```
    def get_price(self):
        return self.__price

    # Setter methods with validation
    def set_price(self, new_price):
        if new_price > 0:
            self.__price = new_price
        else:
            print("Invalid price! Price must be positive.")

    def set_brand(self, new_brand):
        self.__brand = new_brand

    def set_model(self, new_model):
        self.__model = new_model


# Demonstration
car1 = Car("Toyota", "Corolla", 15000)

# Accessing data using getters
print("Car Brand:", car1.get_brand())
print("Car Model:", car1.get_model())
print("Car Price:", car1.get_price())

# Modifying data using setters
car1.set_price(18000)
print("\nUpdated Car Price:", car1.get_price())
```

```
# Trying invalid update
car1.set_price(-5000)  # Will show validation message
```

OUTPUT

```
Car Brand: Toyota
Car Model: Corolla
Car Price: 15000

Updated Car Price: 18000
Invalid price! Price must be positive.
```

Conclusion

- Encapsulation is demonstrated by hiding attributes (, ,) and controlling access through getter and setter methods.
- Direct access to private attributes is restricted, ensuring data security and validation.
- This makes the class more robust and prevents accidental misuse of internal data.

2 Write a class to store and display student details.

Aim

To write a Python program that defines a Student class which stores student details (like name, roll number, marks) and displays them using a method.

⚙ Algorithm

1. Define a class .
2. Create an constructor to initialize attributes (e.g., name, roll number, marks).
3. Define a method to print student details.
4. Create objects of the class with sample data.
5. Call the method to show details.

CODE

```
# Program to store and display student details using a class

class Student:
    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks

    def display(self):
        print("Student Details:")
        print("Name:", self.name)
        print("Roll Number:", self.roll_no)
        print("Marks:", self.marks)



# Creating student objects
s1 = Student("Yogin", 101, 89)
s2 = Student("Priya", 102, 95)

# Displaying student details
s1.display()
print()
s2.display()
```

OUTPUT

```
Student Details:
Name: Yogin
```

Roll Number: 101

Marks: 89

Student Details:

Name: Priya

Roll Number: 102

Marks: 95

Conclusion

- The program defines a Student class with attributes and methods.
- It demonstrates object-oriented programming by storing and displaying student details.
- This can be extended to include more features like calculating grades, storing multiple subjects, or managing a list of students.